R-code

Pooja Panwar[1]
Anant Deshwal[2]

[1]University of Jyväskylä
[2]Bradley University

Credit to:

- Tom Hobbs (Colorado State University)
- Mevin Hooten (University of Texas)
- Becky Tang (Middlebury College)
- Alison Ketz (Funga)
- Slides adopted or based on the 2024 Short course on Bayesian Modeling for Ecologists, funded by NSF (*Award DEB 2042028 to Colorado State University*)

```
##Libraries

library(rjags)  # for running JAGs

library(MCMCvis)  # for checking if your models have converged or not
```

Initial Values

```
inits = list(
  list(K = 1500, r = .2,  sigma = 1),
  list(K = 1000, r = .15, sigma = .1),
  list(K = 900,  r = .3,  sigma = .01))

## You made three MCMC chains. You want the chains to converge
## How do you select what initial values to give to you inits?
   ## You can give them any reasonable value and the chains should ideally converge
   ## Non-reasonable values may cause your model to take longer time to converge
   but it should still converge

   ## Converge means all three MCMC chains so end up in same spot
```

Data

```
data = list(
  n = nrow(Logistic),
  x = as.double(Logistic$PopulationSize),
  y = as.double(Logistic$GrowthRate))
```

Remember your Liklihood model from JAGs. The for loop: for (i in 1:n) {…}. The n is the number of rows. JAGs takes n from here. Loop runs once per observation.

x = as.double(Logistic$PopulationSize): You are telling JAGs here which column in your datasheet represents x (must match name in JAGs model exactly).

y = as.double(Logistic$GrowthRate)): You are telling JAGs here which column in your datasheet represents y (must match name in JAGs model exactly).

Run-Settings

n.adapt = 1000

n.update = 10000

n.iter   = 10000

n.adapt: Tuning steps for tuning the sampler. These are discarded and not valid posterior draws

n.update: burn-in: runs chains for 10,000 more steps and throws away the draws. They represent chain moving from initial values towards high-probability region. They reflect starting values not postrior.

n.iter: Sampling: They represent postrior draws and these values are kept.

21,000 steps taken and only 10,000 kept

Calling JAGs

```
set.seed(1) ## So that results are same everytime you run it
jm = jags.model("LogisticJAGS.R", data = data, inits = inits,
                n.chains = length(inits), n.adapt = n.adapt) ## - Here you specify which file has JAGs model. You give the settings for the run. Telling it where to look for data, MCMC chain initial values and how many values for tuning the sampler

update(jm, n.iter = n.update) ## This is the step where Burn-in happens

zm = coda.samples(jm, variable.names = c("K", "r", "sigma", "tau"),
                  n.iter = n.iter, thin = 1)   ## Tells jm to continue from where Burn-in stopped.
                    ##variable.names is asking it record for the following nodes.
                    ##n.iter- asking to run 10,000 more steps and keep them.
                    #$ thin =1; keep every draw. thin= 10 means keep every 10th draw. Saves memory
                    ##zm is a coda object, returns mcmc.list
```

Calling JAGs

MCMCsummary(zm, round = 3) ## Takes all posterior draws in “zm” and gives one row per parameter

Output (Total 30,000 draws from 3 chains):

```
> MCMCsummary(zm, round = 3)
           mean       sd    2.5%      50%    97.5% Rhat n.eff
K       977.566   26.427 928.949  976.590 1032.630    1 14346
r         0.200    0.008   0.183    0.200    0.216    1 15158
sigma     0.026    0.003   0.021    0.026    0.032    1 15589
tau    1523.194 313.087 971.601 1502.199 2199.877    1 17274
```

Results:

Mean: Mean from 30,000 draws from 3 chains

SD: Postrior SD. Uncertainity in the parameter

2.5% and 97.5%: 95% Credible interval- 95% probability that parameter lies within this range.

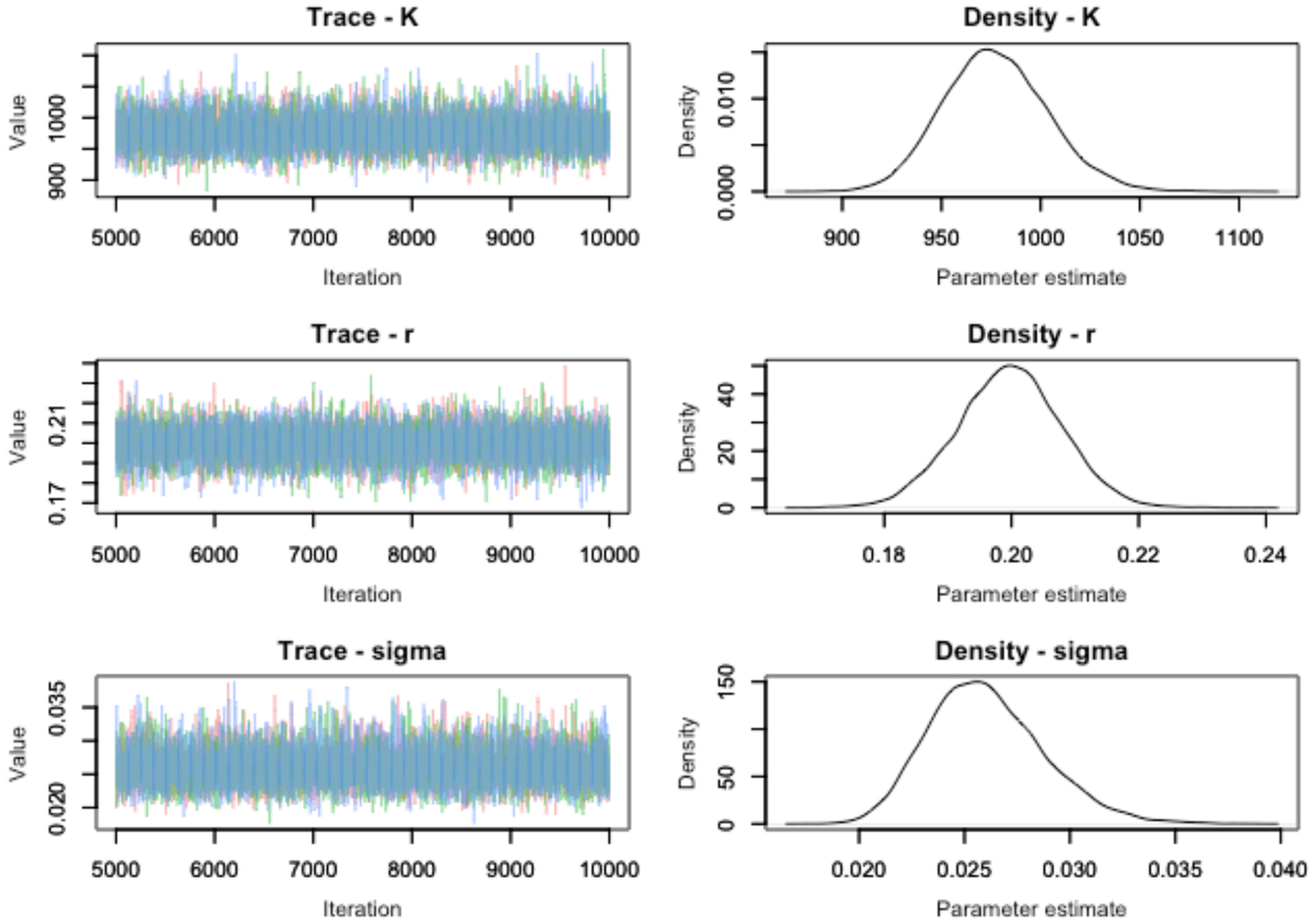
R-hat: One way to check convergence. Important value to report. Above 1.1 means not converged

n.eff: Effective sample size. Draws are autocorrelated. 30,000 draws carry less information than 30,000 independent draws.

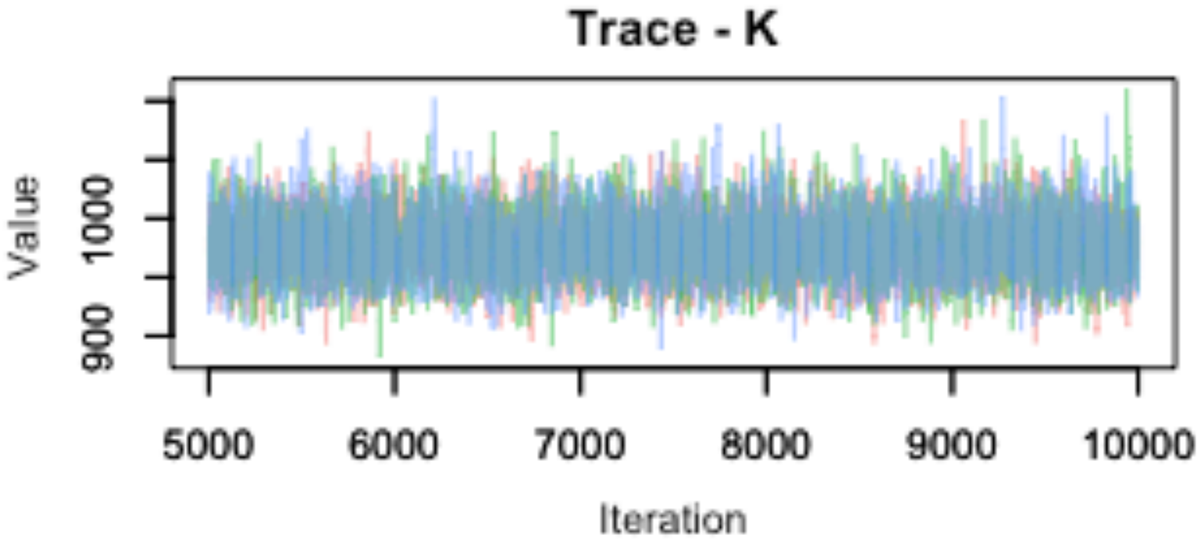
```
> MCMCsummary(zm, round = 3)
          mean      sd    2.5%      50%    97.5% Rhat n.eff
K      977.566  26.427 928.949  976.590 1032.630    1 14346
r        0.200   0.008   0.183    0.200    0.216    1 15158
sigma    0.026   0.003   0.021    0.026    0.032    1 15589
tau   1523.194 313.087 971.601 1502.199 2199.877    1 17274
```

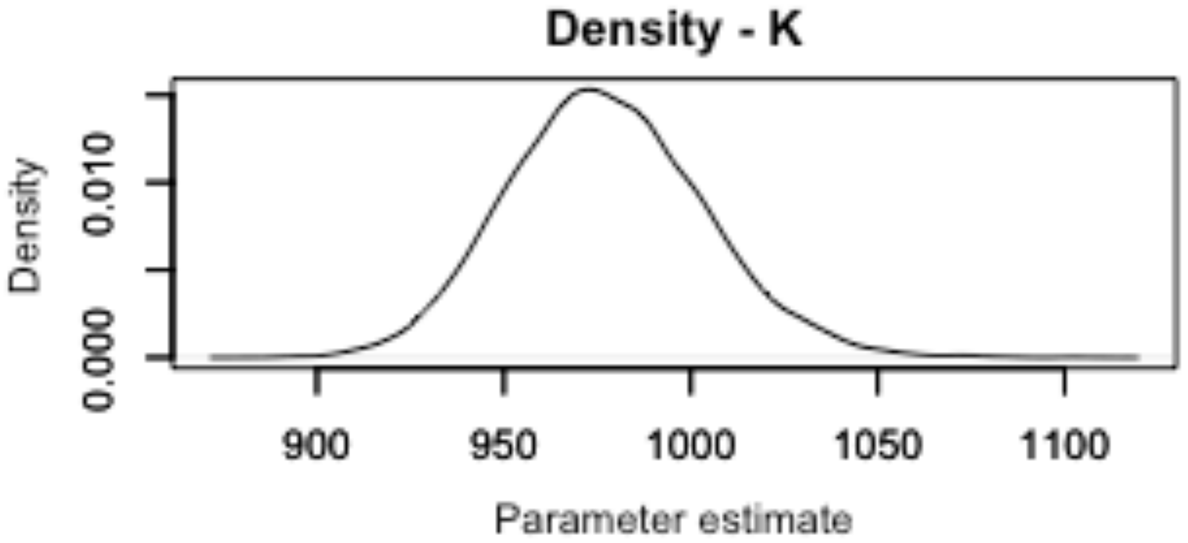
Caterpillar Plots

MCMCtrace(zm, params = c("K", "r", "sigma"), pdf = FALSE) # Calls 2 plots per parameter



Caterpillar Plots





This is what you want to see. Good Hairy Caterpillar. You do not see 3 individual chains. They have all Converged into 1 place.

This is what you want to see. All 3 Curves sit on top of each other. You Do not see three different peaks. 3 Peaks mean that chains found different Answers.